

Fast Iterative Dual-Output-Aware LUT Mapping for Fracturable FPGA Architectures

Author Names Omitted for Review

Abstract—Modern field-programmable gate arrays employ fracturable lookup tables (LUTs), each of which can implement either a single large-input function or two smaller functions within a shared physical LUT site. Conventional technology-mapping flows typically perform dual-output packing only after single-output LUT mapping, thereby preventing potential pairing opportunities from influencing cut selection. This paper presents an iterative dual-output-aware LUT-mapping framework integrated into the delay, area-flow, and exact-area optimization passes of the Berkeley ABC mapper. The proposed method alternates between single-output cut selection and bounded dual-output matching. Candidate pairs are efficiently generated using a sparse support-based index, validated against parameterized architectural, dependency, and output-specific timing constraints, and selected through a heuristic score-based matching procedure. The resulting partner information is then fed back into subsequent mapping rounds through a compatibility-aware cut-cost adjustment. To preserve timing accuracy, the physical union of the inputs is used only for architectural legality checking, while each output retains its own logical timing support. Experiments on the EPFL combinational benchmark suite under two representative architecture models demonstrate average reductions of 34.96% and 23.39% in the reported LUT-area metric relative to vanilla ABC. Compared with the best previously reported method, the proposed framework achieves a $15.8\times$ speedup while further reducing depth by approximately 5% and area by 1%.

Index Terms—FPGA, technology mapping, dual-output LUT, fracturable LUT, timing-aware mapping, LUT packing, logic synthesis.

I. INTRODUCTION

Look-up table (LUT) technology mapping transforms a technology-independent Boolean network into an FPGA implementation [1], [2], [3]. Because mapping determines the number of logic resources and the depth of the resulting network, it directly affects area, timing, power, and routability [4], [5], [6], [7], [8], [9]. Classical methods formulate the problem with K -feasible cuts and optimize delay, area flow, or exact area through iterative mapping rounds [10], [11], [12]. These methods generally assume that each LUT implements one output function.

Modern FPGAs increasingly use fracturable LUTs [13], [14], [15]. A physical LUT site may implement either one large function or two smaller compatible functions that share input resources. Mapping two functions into one site can reduce logic utilization and may also improve local interconnect usage. This opportunity has motivated post-mapping packing, dual-output-aware cut selection, and general multi-output mapping formulations [16], [17], [18], [19], [20].

A common approach first obtains a high-quality single-output cover and then merges compatible LUTs. Although computationally efficient, this strategy is restricted by decisions made before packing. Two roots may have alternative

cuts that form a beneficial pair, but these cuts may never be selected when each root is optimized independently. At the other extreme, explicit multi-output cut enumeration exposes more alternatives but substantially increases the search space and runtime. A practical mapper must therefore balance mapping quality against candidate-generation and matching cost. Existing general multi-output formulations provide a principled search model [18], [21], but their larger cut spaces and more complex cost evaluation can limit their benefit on large networks. This motivates a narrower integration in which the well-established single-output mapper remains responsible for cut enumeration, while a separate bounded stage identifies physical sharing opportunities and returns only their estimated benefit to later recovery rounds.

Timing evaluation introduces a second difficulty. The outputs of a fracturable LUT share a physical input interface, but they do not necessarily have identical logical dependencies. Using the union of both supports as the timing support of each output creates false timing arcs. A dual-output mapper must distinguish physical input compatibility from output-specific logical timing support.

This work couples conventional cut selection with bounded dual-output pairing. After each area-recovery round, the selected cuts form a sparse compatibility graph, where legal and timing-feasible pairs are scored and matched. The estimated site benefit of committed pairs is then fed back into subsequent cut-selection rounds, favoring cuts that are both individually competitive and jointly compatible. This approach exposes site-sharing opportunities before the final cover is fixed while preserving the scalability of priority-cut mapping. Although heuristic and sensitive to candidate bounds, the initial cover, and graph retention, it recovers useful dual-output opportunities without exhaustive multi-output cut enumeration.

The main contributions are as follows:

- An iterative mapping framework that feeds dual-output compatibility and physical-site benefit into subsequent single-output cut-selection rounds.
- A parameterized legality model that separates per-output support, physical input-union capacity, optional sharing constraints, root independence, and output-specific timing support.
- A bounded shared-input candidate search and sparse matching procedure that controls runtime while retaining high-value pairing opportunities.

II. BACKGROUND

A. Fracturable and Dual-Output LUTs

Modern FPGAs are commonly organized as arrays of configurable logic blocks (CLBs) connected by programmable

routing resources. As shown in Fig. 1(a), a CLB contains multiple LUT sites, local input routers, flip-flops, output multiplexers, and interfaces to the routing fabric through connection and switch blocks. The external routing channels deliver signals to the CLB input pins, denoted by I , and the local routers select up to six inputs for each LUT. The parameter N denotes the number of LUT sites in the CLB, and since each LUT site can provide two output paths, the CLB can expose up to $2N$ local outputs. Each LUT output can either bypass or pass through a D flip-flop (DFF) driven by CLK before entering the connection block. Therefore, LUT mapping affects not only the number of used LUT sites, but also local output usage, sequential resource usage, and routing pressure inside the FPGA fabric.

A key feature of commercial FPGA LUTs is fracturability. A fracturable LUT can operate either as one large single-output LUT or as two smaller LUTs sharing part of their input set. Fig. 1(b) shows a representative dual-output LUT6 structure. Inputs $A_1, \dots, A_6$ are available to the physical LUT. The upper output D_6 can implement a six-input function, while the lower output D_5 can expose one five-input subfunction. In dual-output mode, the structure can be viewed as two LUT5 blocks that share inputs $A_2, \dots, A_6$. The signal A_1 is used to select between the two LUT5 outputs for the D_6 output through the internal multiplexer. Meanwhile, one of the LUT5 outputs can be directly observed at D_5 .

Commercial FPGA families commonly exploit this type of structure [13], [3]. For example, a LUT6 can implement either one six-input function or two five-input functions when the two functions share the required input subset. This is the basic dual-output mode used in many LUT6-based architectures. Some newer FPGA architectures provide more flexible fracturable LUT resources [22], [23], [24], [25], [26], allowing two outputs with larger compatible input supports, but the same principle remains: multiple logic functions may share one physical LUT only when their individual supports and their union support satisfy the architectural constraints.

We describe a target architecture by

$$\mathcal{A} = (K_{\text{single}}, K_{\text{out}}, K_{\text{union}}, S_{\text{min}}, S_{\text{max}}), \quad (1)$$

where K_{single} is the single-output input limit, K_{out} is the support limit of each function in dual-output mode, K_{union} is the physical input-union limit, and S_{min} and S_{max} optionally bound the number of shared inputs. When an architecture does not impose an explicit sharing constraint, we use $S_{\text{min}} = 0$ and $S_{\text{max}} = K_{\text{out}}$.

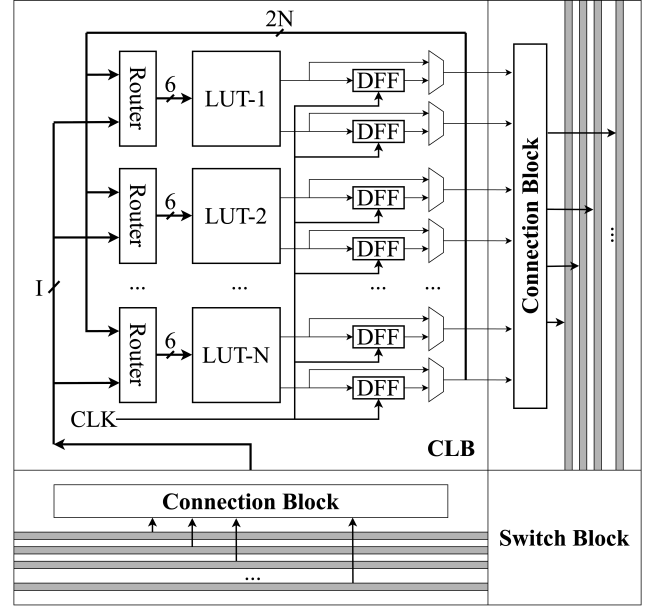
For candidate cuts C_u and C_v , let $I_u = \text{supp}(C_u)$ and $I_v = \text{supp}(C_v)$. The pair is architecture legal only if

$$\begin{aligned} |I_u| &\leq K_{\text{out}}, & |I_v| &\leq K_{\text{out}}, \\ |I_u \cup I_v| &\leq K_{\text{union}}, & S_{\text{min}} &\leq |I_u \cap I_v| \leq S_{\text{max}}. \end{aligned} \quad (2)$$

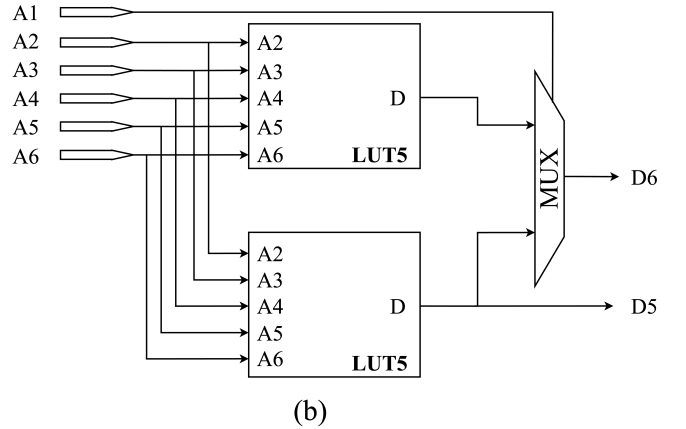
The roots must also be mutually independent. If r_u and r_v are the roots of C_u and C_v , a legal pair must satisfy

$$r_u \notin I_v, \quad r_v \notin I_u. \quad (3)$$

These parameterized constraints allow the same mapping formulation to target different fracturable FPGA architectures.



(a)



(b)

Fig. 1. (a) FPGA configurable logic block and (b) representative fracturable LUT6 dual-output structure.

B. K -Feasible Cut Enumeration for Single-Output LUT

Traditional LUT technology mapping in ABC is based on K -feasible cut enumeration [10], [11], [12]. Given a combinational network represented as a directed acyclic graph $G = (V, E)$, each internal node $v \in V$ is treated as a possible LUT root. A cut C_v of node v is a set of nodes that separates v from the primary inputs. Equivalently, every path from any primary input to v must pass through at least one node in C_v . If the cut contains no more than K nodes, it is called a K -feasible cut and can be implemented by one K -input LUT:

$$|C_v| \leq K. \quad (4)$$

Fig. 2 illustrates several feasible cuts rooted at the same logic node. The dashed nodes at the top, labeled $a-f$, represent primary inputs or previously mapped signals. The solid nodes represent internal logic nodes in the transitive fanin cone of the target root. Strictly, a cut is the boundary node set that separates the root from the primary inputs; the shaded regions in the figure illustrate the logic cones induced by

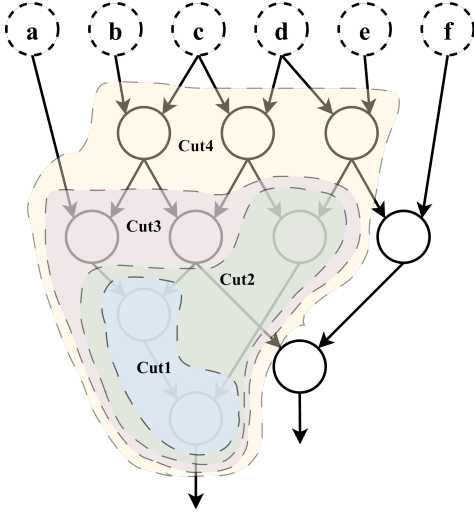


Fig. 2. Illustration of multiple K -feasible cuts rooted at the same logic node in conventional single-output LUT mapping. The shaded regions denote cut-induced logic cones, while each cut is formally defined by its boundary leaf set.

different candidate cuts. For example, Cut_1 represents a small local cone close to the root, while Cut_4 represents a larger cone that reaches farther toward the primary inputs. These alternative cut-induced cones expose different possible LUT implementations for the same root node.

During cut enumeration, cuts are generated bottom-up following the topological order of the network. For a two-input logic node v with fanins u_0 and u_1 , candidate cuts of v are produced by merging cuts from its fanins, retaining only K -feasible merged cuts; the trivial cut may also be considered:

$$C_v = C_{u_0} \cup C_{u_1}, \quad |C_{u_0} \cup C_{u_1}| \leq K, \quad C_v = \{v\}. \quad (5)$$

This procedure produces multiple candidate cuts for each node, corresponding to different possible LUT boundaries.

Because the number of feasible cuts can grow rapidly, practical mappers keep only a bounded set of priority cuts for each node. Let $\mathcal{C}(v)$ denote the set of retained cuts for node v . ABC limits its size by a parameter n_{cuts} , such that $|\mathcal{C}(v)| \leq n_{\text{cuts}}$. The retained cuts are ranked according to mapping objectives such as delay, area flow, exact area, edge count, or tie-breaking cost. Therefore, cut enumeration is not only a feasibility procedure, but also the foundation for optimization: different retained cuts expose different trade-offs between logic depth and resource usage.

For each feasible cut, the mapper computes timing and area-related metrics. The delay of a cut is determined by the latest arrival time among its leaves plus the delay of the LUT implementing the cut:

$$\text{arr}(v, C_v) = \max_{u \in C_v} \text{arr}(u) + d_{\text{LUT}}(C_v). \quad (6)$$

A delay-oriented mapping round selects cuts that minimize the arrival time of each node. After the initial delay mapping, ABC performs area-flow and exact-area recovery rounds. Area flow estimates the fractional contribution of a cut by distributing the cost of shared fanin logic across its fanouts, while exact-

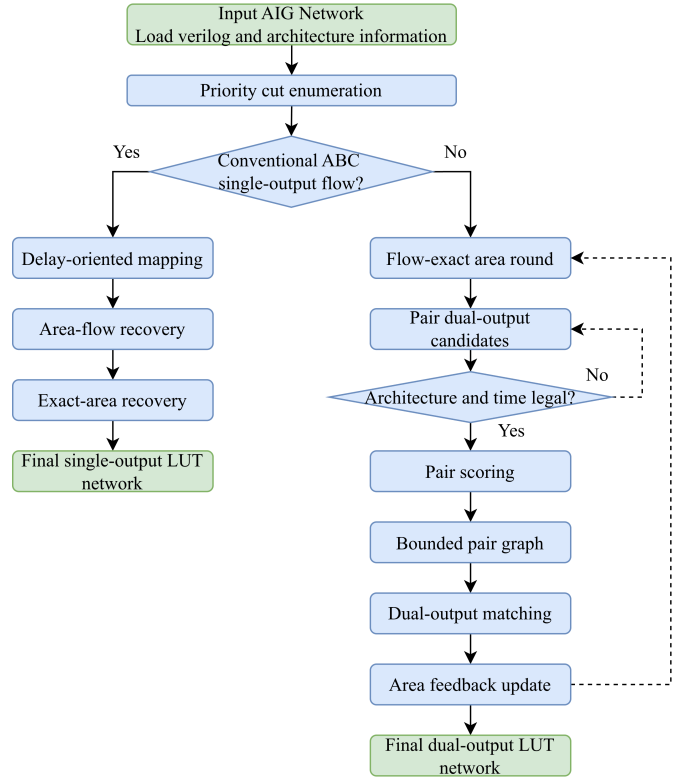


Fig. 3. Overview of the proposed iterative dual-output-aware mapping flow compared with the conventional ABC single-output mapping flow.

area recovery evaluates the incremental area more directly by temporarily dereferencing and referencing mapped cones.

Finally, for conventional single-output mapping, one best cut is selected for each mapped root. The selected cut defines one single-output LUT whose input pins are the leaves of the cut and whose output is the root node. Thus, the standard ABC mapping flow constructs a network of single-output LUTs, $v \leftarrow \text{LUT}(C_v)$, where each LUT implements one Boolean function and satisfies the K -input constraint in Eq. (4). This single-output cut enumeration and selection process is the baseline on which the proposed dual-output mapping method is built.

III. PROPOSED METHOD

A. Overview of Cut-based Mapping Flow

Fig. 3 summarizes the proposed dual-output-aware mapping flow and its relationship to the conventional ABC single-output mapper. The input network is first converted into an and-inverter graph (AIG), and architecture information is loaded to specify the LUT capacity and dual-output legality constraints. The mapper then performs priority cut enumeration, which provides the candidate single-output cuts used by both the conventional and proposed flows.

In the conventional ABC flow, the mapper proceeds linearly after cut enumeration. It first performs delay-oriented mapping to obtain a timing-driven initial cover, then applies area-flow recovery and exact-area recovery to reduce LUT usage under timing constraints. The final result is a single-output LUT

network in which each mapped LUT implements one output function.

The proposed flow keeps the ABC cut enumeration and mapping rounds, but embeds dual-output matching into the iterative mapping process. After each flow or exact-area round, the currently referenced mapped roots are treated as single-output groups. The algorithm then constructs candidate dual-output pairs from these groups. Candidate generation is guided by shared cut inputs: LUTs that use common input nodes are placed into shared-input buckets, and only a bounded number of related candidates are evaluated. This avoids exhaustive all-pair enumeration while still focusing the search on pairs that are likely to share a physical fracturable LUT.

Each generated pair is then checked against the architecture and timing constraints. A pair is rejected if either output cut exceeds the dual-output input limit, if the union of the two input sets exceeds the physical LUT input limit, if one root depends on the other root, or if the output-specific timing constraint is violated. Illegal pairs are discarded, and the algorithm returns to candidate generation for additional candidates. Legal pairs are scored according to their expected area saving, shared-input quality, timing impact, and structural complexity.

The legal and scored pairs form a sparse bounded pair graph. The mapper first commits mutual-best pairs, where both endpoints select each other as their best partner. It then greedily commits the remaining non-conflicting pairs in descending score order, ensuring that each mapped root participates in at most one dual-output group. After matching, the committed partner relations are stored and used as feedback in the next mapping round. Specifically, cuts that remain compatible with previously selected dual-output partners receive a reduced area cost, encouraging subsequent ABC mapping rounds to preserve or improve useful dual-output opportunities.

This feedback loop couples single-output cut selection with dual-output LUT packing. Therefore, the proposed method is not merely a post-processing merge of an already mapped single-output network. Instead, dual-output feasibility and area benefit are iteratively propagated back into the cut-selection process. The final output is a dual-output LUT network in which matched pairs share physical fracturable LUT resources, while unmatched roots remain as ordinary single-output LUTs.

B. Bounded Candidate Generation

A direct way to identify dual-output LUT pairs is to test every pair of currently referenced LUTs. However, this all-pair strategy is expensive. If there are m referenced LUTs in the mapped network, exhaustive pair checking requires $\mathcal{O}(m^2)$ candidate evaluations. Most of these pairs are unlikely to be useful because pairs with common inputs tend to have a smaller physical input union. Therefore, the proposed mapper uses heuristic bounded pruning based on shared-input buckets.

Fig. 4 illustrates the basic idea. Each currently referenced LUT is first represented by its input support. For example, the target LUT-A uses $I_A = \{x_1, x_2, x_3, x_4\}$. Instead of comparing LUT-A with all other LUTs, the mapper builds

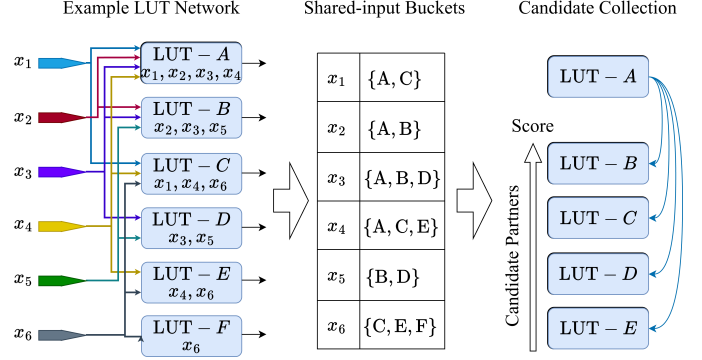


Fig. 4. Shared-input bucket based bounded pruning for dual-output LUT candidate generation. LUTs are indexed by their input supports, and only LUTs sharing inputs with the target LUT are collected as candidate partners before legality checking and scoring.

an inverse index from each input signal to the LUTs that use this input:

$$\mathcal{B}(x_i) = \{L \mid x_i \in I_L\}, \quad (7)$$

where $\mathcal{B}(x_i)$ is the shared-input bucket of input x_i and I_L is the input support of LUT L .

For the example in Fig. 4, the buckets associated with the inputs of LUT-A are

$$\begin{aligned} \mathcal{B}(x_1) &= \{A, C\}, & \mathcal{B}(x_2) &= \{A, B\}, \\ \mathcal{B}(x_3) &= \{A, B, D\}, & \mathcal{B}(x_4) &= \{A, C, E\}. \end{aligned} \quad (8)$$

The raw candidate partner set of LUT-A is then obtained by taking the union of these buckets and removing A itself:

$$\mathcal{P}_A = \left(\bigcup_{x_i \in I_A} \mathcal{B}(x_i) \right) \setminus \{A\} = \{B, C, D, E\}. \quad (9)$$

This bucket-based construction omits pairs that do not share any input with the target LUT. For instance, LUT-F is not considered as a candidate partner for LUT-A because it only uses x_6 , which is not in I_A . In this way, the mapper avoids evaluating pairs before applying the more expensive architecture and timing legality checks. Shared input, however, is not a necessary condition for legality when $S_{\min} = 0$: two disjoint supports may still satisfy $|I_u \cup I_v| \leq K_{\text{union}}$. Consequently, the current generator is heuristic and does not enumerate every architecture-legal pair. In this paper, bounded candidate pruning denotes this quality-runtime trade-off; it is not an exact elimination rule and provides no completeness guarantee.

To control runtime, two search limits are introduced. The first limit, n_{bucket} , bounds the number of entries scanned from each shared-input bucket. The second limit, n_{cand} , bounds the number of distinct candidate partners evaluated for each target LUT:

$$|\text{scan}(\mathcal{B}(x_i))| \leq n_{\text{bucket}}, \quad |\mathcal{P}_A^{\text{eval}}| \leq n_{\text{cand}}. \quad (10)$$

These two parameters correspond to the implementation options $n_{\text{BucketScanMax}}$ and $n_{\text{CandidatesMax}}$, respectively.

Algorithm 1 gives the corresponding implementation. For bounded support size K , bucket construction costs $\mathcal{O}(mK)$

Algorithm 1 Bounded Shared-Input Candidate Generation

```

1: Input: referenced LUT roots  $R$ , supports  $I_u$ , limits  $n_{\text{bucket}}, n_{\text{cand}}$ 
2: Output: candidate partner sets  $\mathcal{P}_u$ 
3: for each input  $x$  do
4:    $\mathcal{B}(x) \leftarrow \emptyset$ 
5: end for
6: for each root  $u \in R$  do
7:   for each input  $x \in I_u$  do
8:     Insert  $u$  into  $\mathcal{B}(x)$ 
9:   end for
10: end for
11: for each root  $u \in R$  do
12:    $\mathcal{P}_u \leftarrow \emptyset$ 
13:   for each input  $x \in I_u$  do
14:     Scan at most  $n_{\text{bucket}}$  entries of  $\mathcal{B}(x)$ 
15:     for each scanned root  $v$  do
16:       if  $v \neq u$  then
17:         Insert  $v$  into  $\mathcal{P}_u$  if it is new
18:       end if
19:       if  $|\mathcal{P}_u| = n_{\text{cand}}$  then
20:         Stop collecting candidates for  $u$ 
21:       end if
22:     end for
23:   end for
24: end for
25: return  $\{\mathcal{P}_u \mid u \in R\}$ 

```

and candidate scanning costs $\mathcal{O}(mKn_{\text{bucket}})$, capped by $\mathcal{O}(mn_{\text{cand}})$ distinct evaluated candidates [27].

After candidate collection, the remaining candidates are not immediately merged. Each candidate pair is first checked against the dual-output legality constraints, including individual cut-size limits, union input limits, root-independence constraints, and output-specific timing validity. Legal pairs are then scored. A simplified score for a target LUT u and a candidate LUT v , consistent with the detailed scoring model in Section III-E, can be written as

$$S(u, v) = \alpha \Delta A(u, v) + \beta |I_u \cap I_v| - \eta |I_u \cup I_v| - \gamma \Delta D(u, v), \quad (11)$$

where $\Delta A(u, v)$ is the estimated physical LUT area saving, $|I_u \cap I_v|$ rewards shared inputs, $|I_u \cup I_v|$ penalizes larger merged input support, and $\Delta D(u, v)$ captures timing degradation or timing risk. The coefficients α , β , η , and γ control the relative importance of these terms.

Finally, only high-scoring legal pairs are retained in the bounded pair graph. For each LUT, the mapper keeps at most n_{pair} candidate edges, $|\mathcal{E}_u| \leq n_{\text{pair}}$, where $\mathcal{E}_u$ denotes the retained legal pair edges incident to LUT u . This final pruning step further reduces matching complexity while preserving the most promising dual-output opportunities.

C. Pair-Compatibility Feedback to Cut Selection

Let $\mathcal{M}^{(t-1)}$ be the nonconflicting pair set produced after the previous recovery round, and let $p^{(t-1)}(u)$ denote the partner

Algorithm 2 Iterative Dual-Output-Aware Mapping

```

1: Input: AIG  $G$ , architecture  $\mathcal{A}$ , recovery-round budget  $T$ 
2: Output: mapped LUT network and final pair set  $\mathcal{M}$ 
3: Enumerate priority cuts and run the original delay-oriented mapping
4:  $\mathcal{M} \leftarrow \emptyset$ 
5: for  $t = 1$  to  $T$  over area-flow and exact-area rounds do
6:   Filter cuts using the original ABC delay constraint
7:   Rank feasible cuts using Eq. (13), as applicable
8:   Select one cut for each referenced root
9:   Generate bounded candidates and discard illegal pairs
10:  Score legal edges and compute a nonconflicting set  $\mathcal{M}'$ 
11:   $\mathcal{M} \leftarrow \mathcal{M}'$ 
12: end for
13: Recheck  $\mathcal{M}$  against the final selected cuts
14: Commit valid pairs; retain every other root as a single-output LUT
15: return mapped network and  $\mathcal{M}$ 

```

of root u in that set. For a candidate cut C_u considered in round t , define

$$\chi(C_u, p^{(t-1)}(u)) = \begin{cases} 1, & \text{if } C_u \text{ is legal with cut of } p^{(t-1)}(u), \\ 0, & \text{otherwise.} \end{cases} \quad (12)$$

The estimated benefit of retaining that compatibility and the resulting feedback-adjusted area-flow and exact-area ranking costs are

$$\begin{aligned} B(C_u, C_p) &= \max(0, A(C_u) + A(C_p) - A_{\text{dual}}(C_u, C_p)), \\ \text{AF}_{\text{fb}}(C_u) &= \text{AF}_{\text{ABC}}(C_u) - \chi(C_u, p^{(t-1)}(u))B(C_u, C_p), \\ \text{EA}_{\text{fb}}(C_u) &= \text{EA}_{\text{ABC}}(C_u) - \chi(C_u, p^{(t-1)}(u))B(C_u, C_p). \end{aligned} \quad (13)$$

The original ABC delay constraint is applied before this adjustment, so feedback changes only the ranking of delay-feasible cuts and cannot make an illegal or delay-infeasible cut selectable.

Algorithm 2 summarizes the outer loop. The current implementation terminates after the predefined ABC area-flow and exact-area recovery-round budgets are exhausted. All pairs are checked again against the final cuts before they are committed.

D. Output-Specific Timing After Dual-Output Pairing

Dual-output LUT pairing changes the physical implementation of two mapped cuts, but it should not change the logical dependency of each output. This is an important distinction for timing analysis. Fig. 5 illustrates the issue. Before pairing, the two single-output LUT cones are independent. The first output, out_0 , depends on its own cut input set, while the second output, out_1 , depends on another cut input set. After pairing, the two cuts are placed into one shared physical LUT structure whose input pins correspond to the union of the two input sets. However, this physical union does not mean that every union input becomes a timing predecessor of both outputs.

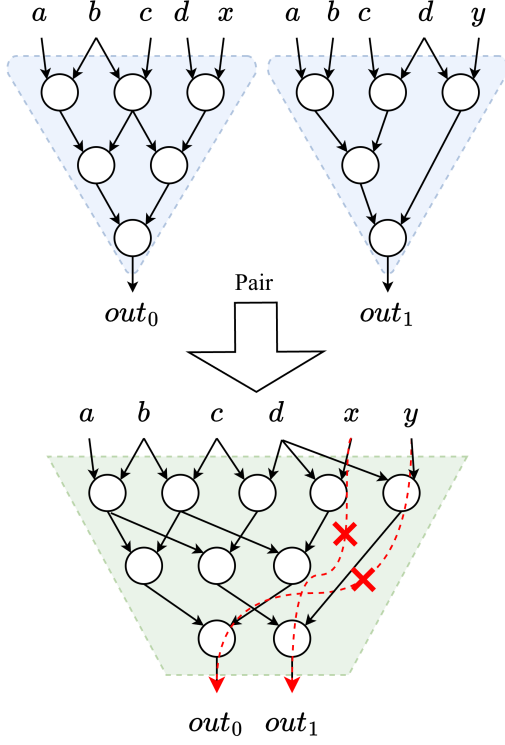


Fig. 5. Output-specific timing interpretation after dual-output LUT pairing. The merged physical LUT receives the union of the two input sets, but each output is timed only through its own original cut. Red dashed paths indicate artificial timing paths that should not be introduced.

Let the two mapped cuts be C_u and C_v , rooted at outputs r_u and r_v , respectively. Their input supports and the merged physical input set are

$$I_u = \text{supp}(C_u), \quad I_v = \text{supp}(C_v), \quad I_{uv} = I_u \cup I_v. \quad (14)$$

The set I_{uv} is used only to check whether the pair fits into the target fracturable LUT architecture. It should not be interpreted as the timing support of both outputs.

For correct timing evaluation, each output must be timed through its own original cut. Therefore, the arrival times are computed as

$$\begin{aligned} \text{arr}(r_u) &= \max_{p \in I_u} \text{arr}(p) + d_u(C_u), \\ \text{arr}(r_v) &= \max_{p \in I_v} \text{arr}(p) + d_v(C_v). \end{aligned} \quad (15)$$

Here, $d_u(C_u)$ and $d_v(C_v)$ denote the output-specific LUT delays for the two outputs. These delays may be identical in a simplified LUT model, but they are written separately to allow architecture-dependent output delays.

A common incorrect interpretation is to time both outputs using the full union input set I_{uv} . This would imply

$$\begin{aligned} \text{arr}(r_u) &= \max_{p \in I_{uv}} \text{arr}(p) + d_u(C_u), \\ \text{arr}(r_v) &= \max_{p \in I_{uv}} \text{arr}(p) + d_v(C_v). \end{aligned} \quad (16)$$

However, this creates artificial timing paths. For example, an input that belongs only to I_v would be treated as a predecessor

Algorithm 3 Dual-Output Legality and Timing Check

```

1: Input: pair  $(u, v)$ , cuts  $C_u, C_v$ , roots  $r_u, r_v$ , required
   times  $\text{req}(r_u), \text{req}(r_v)$ 
2: Output: legal or illegal
3:  $I_u \leftarrow \text{supp}(C_u), I_v \leftarrow \text{supp}(C_v)$ 
4: if  $|I_u| > K_{\text{out}}$  or  $|I_v| > K_{\text{out}}$  then
5:   return illegal
6: end if
7: if  $|I_u \cup I_v| > K_{\text{union}}$  then
8:   return illegal
9: end if
10: if  $|I_u \cap I_v| < S_{\text{min}}$  or  $|I_u \cap I_v| > S_{\text{max}}$  then
11:   return illegal
12: end if
13: if  $r_u \in I_v$  or  $r_v \in I_u$  then
14:   return illegal
15: end if
16: Compute  $\text{arr}(r_u)$  and  $\text{arr}(r_v)$  with output-specific sup-
   ports
17: if  $\text{arr}(r_u) > \text{req}(r_u)$  or  $\text{arr}(r_v) > \text{req}(r_v)$  then
18:   return illegal
19: end if
20: return legal

```

of r_u , and an input that belongs only to I_u would be treated as a predecessor of r_v . These paths do not exist in the original Boolean functions and are marked as invalid in Fig. 5.

Therefore, the proposed mapper separates physical compatibility from timing dependency. The union set $I_u \cup I_v$ is used for architecture legality, while timing is evaluated using the output-specific supports I_u and I_v . The pair is timing legal only if both outputs satisfy their required times:

$$\text{arr}(r_u) \leq \text{req}(r_u), \quad \text{arr}(r_v) \leq \text{req}(r_v). \quad (17)$$

This preserves the original logical timing dependencies under the evaluated delay model while allowing the mapper to exploit the estimated site benefit of sharing one dual-output LUT.

Algorithm 3 summarizes the complete legality test. It combines the architectural constraints in Eqs. (2)–(3) with the output-specific timing conditions in Eqs. (15) and (17). For bounded LUT size, the test costs $\mathcal{O}(K)$ per candidate pair.

E. Dual-Output Pair Scoring

After bounded candidate generation and architecture legality checking, the mapper obtains a set of legal dual-output candidate pairs. These pairs are not immediately committed, because one LUT can participate in at most one dual-output group. Therefore, the mapper assigns a score to each legal pair and constructs a bounded pair graph for the subsequent matching stage.

Let u and v be two currently referenced mapped roots, with selected cuts C_u and C_v . Their input supports and merged support are

$$I_u = \text{supp}(C_u), \quad I_v = \text{supp}(C_v), \quad I_{uv} = I_u \cup I_v. \quad (18)$$

The number of shared inputs is

$$n_{\text{shared}}(u, v) = |I_u| + |I_v| - |I_{uv}|. \quad (19)$$

The primary objective of pairing is to reduce the estimated occupied LUT-site count. Thus, the first term in the score is the estimated area saving obtained by implementing u and v using one dual-output LUT instead of two separate single-output LUTs:

$$\Delta A(u, v) = A(C_u) + A(C_v) - A_{\text{dual}}(I_{uv}) > 0. \quad (20)$$

where $A(C_u)$ and $A(C_v)$ are the costs of the two individual LUTs, and $A_{\text{dual}}(I_{uv})$ is the physical cost of the merged dual-output LUT with input set I_{uv} .

In addition to area saving, the score favors pairs with stronger input sharing. A larger shared input count usually indicates a more natural fracturable LUT packing opportunity, because the two functions use similar signals and are less likely to increase local routing pressure. The score also penalizes large merged supports, since a pair using many distinct inputs consumes more of the physical LUT input capacity and may reduce architectural flexibility.

Timing is incorporated as a penalty term. Let $\text{arr}_{\text{orig}}(u)$ and $\text{arr}_{\text{orig}}(v)$ denote the arrival times before pairing, and let $\text{arr}_{\text{pair}}(u)$ and $\text{arr}_{\text{pair}}(v)$ denote the output-specific arrival times after pairing. The timing penalty is defined as

$$\Delta D(u, v) = \max \left(0, \max (\text{arr}_{\text{pair}}(u), \text{arr}_{\text{pair}}(v)) - \max (\text{arr}_{\text{orig}}(u), \text{arr}_{\text{orig}}(v)) \right). \quad (21)$$

Pairs that violate required times are discarded before scoring:

$$\text{arr}_{\text{pair}}(u) \leq \text{req}(u), \quad \text{arr}_{\text{pair}}(v) \leq \text{req}(v). \quad (22)$$

The final score combines area saving, shared-input reward, timing penalty, and structural complexity penalty:

$$S(u, v) = \alpha \Delta A(u, v) + \beta n_{\text{shared}}(u, v) - \gamma \Delta D(u, v) - \eta |I_{uv}| - \theta |N_{uv}|. \quad (23)$$

where N_{uv} is the set of internal nodes covered by the merged pair. The coefficients α , β , γ , η , and θ control the relative importance of area reduction, input sharing, timing preservation, input usage, and structural complexity.

In the implementation, area saving is assigned the largest weight because the main purpose of dual-output pairing is to reduce the estimated occupied LUT-site count. The shared-input term then breaks ties among pairs with similar area saving, while timing and complexity penalties prevent the mapper from selecting pairs that are structurally large or risky for timing. Legal pairs are sorted in descending score order. For each LUT, only the best bounded number of incident edges is retained, $|\mathcal{E}_u| \leq n_{\text{pair}}$, where $\mathcal{E}_u$ is the retained pair-edge set incident to LUT u and n_{pair} corresponds to the implementation parameter $n_{\text{PairsPerObjMax}}$.

The resulting bounded pair graph is then passed to the matching stage. The mapper first commits mutual-best pairs and then greedily commits remaining non-conflicting pairs in descending score order. This score-based filtering keeps the matching problem small while preserving the most promising dual-output LUT packing opportunities.

Algorithm 4 Pair Scoring and Matching

```

1: Input: candidate sets  $\mathcal{P}_u$ , selected cuts, limit  $n_{\text{pair}}$ 
2: Output: committed pair set  $\mathcal{M}$ 
3: for each root  $u$  do
4:    $\mathcal{E}_u \leftarrow \emptyset$ 
5:   for each candidate  $v \in \mathcal{P}_u$  do
6:     if Algorithm 3 returns legal for  $(u, v)$  then
7:       Compute  $\Delta A(u, v)$ ,  $\Delta D(u, v)$ , and  $S(u, v)$ 
8:       if  $\Delta A(u, v) > 0$  then
9:         Insert  $(u, v, S(u, v))$  into  $\mathcal{E}_u$ 
10:      end if
11:    end if
12:  end for
13:  Retain the top  $n_{\text{pair}}$  edges in  $\mathcal{E}_u$ 
14: end for
15:  $\mathcal{E} \leftarrow \bigcup_u \mathcal{E}_u$  sorted by descending score
16:  $\mathcal{M} \leftarrow \emptyset$  and mark all roots unmatched
17: for each edge  $(u, v) \in \mathcal{E}$  do
18:   if  $(u, v)$  is the best edge of both endpoints and both are
    unmatched then
19:     Add  $(u, v)$  to  $\mathcal{M}$ ; mark  $u$  and  $v$  matched
20:   end if
21: end for
22: for each edge  $(u, v) \in \mathcal{E}$  in descending score order do
23:   if  $u$  and  $v$  are both unmatched then
24:     Add  $(u, v)$  to  $\mathcal{M}$ ; mark  $u$  and  $v$  matched
25:   end if
26: end for
27: return  $\mathcal{M}$ 

```

TABLE I
REPRESENTATIVE FRACTURABLE-LUT ARCHITECTURE MODELS.
NEITHER MODEL IMPOSES AN ADDITIONAL INPUT-SHARING CONSTRAINT.

Architecture	K_{single}	K_{out}	K_{union}	S_{min}	S_{max}
ultrascale+	6	5	5	0	5
Versal	6	6	6	0	6

Algorithm 4 gives the score-based matching procedure. If q candidates are evaluated and e legal edges are retained, scoring costs $\mathcal{O}(qK)$, sorting costs $\mathcal{O}(e \log e)$, and the two matching passes cost $\mathcal{O}(e)$.

IV. EXPERIMENTAL SETUP

This section outlines the evaluation methodology, including the hardware and software verification platform, the target fracturable LUT architectures, the parameter-scan configurations, and the benchmark suites along with their respective evaluation metrics.

A. Experimental Platform

To evaluate the performance of the proposed dual-output-aware LUT mapping framework, all algorithms are implemented in C++ within the ABC synthesis environment. The experimental evaluations are executed on a 64-bit Windows

TABLE II
TAXONOMY, BASELINE SETTINGS, AND SCAN RANGES OF MAPPING PARAMETERS.

Category	Parameter	Description	Baseline	Scan Range / Set
Conventional ABC	K	LUT size	6	—
	n_F	Number of area-flow rounds	1	{0, 1, 2, ..., 9} (Joint)
	n_A	Number of exact-area rounds	5	
	n_{cuts}	Max cuts retained per node	24	{1, 2, 4, 6, 8, 12, 16, 24, 32, 48, 64, 96, 128}
Dual-Output Specific	n_{bucket}	Bucket scan limit	16	{4, 8, 16, 32, 64} (Joint)
	n_{cand}	Candidate limit	24	{4, 8, 16, 32, 64} (Joint)
	n_{pair}	Retained pair limit per object	16	{2, 4, 8, 16, 32, 64}

workstation powered by a 12th Gen Intel(R) Core(TM) i7-12700K processor operating at 3.60 GHz, equipped with 32.0 GB of physical memory.

B. Target FPGA Architectures

We evaluate two representative architecture models rather than claim an exact implementation of a particular commercial device. Both allow six inputs in single-output mode. The restricted model limits each dual-output function and their physical union to five inputs, whereas the Versal-like model permits up to six inputs for each quantity. Neither model adds a nontrivial sharing constraint, so $S_{\min} = 0$ and $S_{\max} = K_{\text{out}}$. Legality follows Eqs. (2)–(3). The labels “UltraScale-like” and “Versal-like” retained inside the generated plots correspond to the UltraScale-like and Versal-like abstract models, respectively; they are plot identifiers and should not be interpreted as a complete model of those commercial devices.

C. Mapping Parameters and Scan Ranges

The proposed mapper exposes both conventional ABC mapping parameters and dual-output-specific knobs. To systematically characterize their algorithmic impacts, we establish a robust baseline configuration and execute multi-dimensional individual or joint parameter sweeps. The absolute taxonomy, baseline settings, and empirical scan intervals are encapsulated in Table II. During any isolated parameter sweep, all non-scanned parameters are strictly locked at their default baseline values to isolate variables.

D. Benchmarks and Evaluation Metrics

Our evaluation utilizes standard combinational designs from the EPFL benchmark suite. The reference baseline is generated by running the vanilla, single-output ABC mapping flow to output a standard BLIF representation. The proposed dual-output-aware mapper is then driven by the same source netlists under varying parameter configurations.

For an integer site-count evaluation, let N_{single} be the number of unpaired LUT roots and N_{dual} the number of committed root pairs. The occupied physical-site count and dual-output-site utilization are

$$N_{\text{phys}} = N_{\text{single}} + N_{\text{dual}}, \quad R_{\text{dual-site}} = \frac{N_{\text{dual}}}{N_{\text{phys}}} \times 100\%. \quad (24)$$

The source result image reports some LUT-area values at half-site precision [28]. To provide an integer comparison table, those values are rounded to the nearest integer in Tables III and IV; percentages and depths are copied unchanged. The resulting N_{prop} column is therefore an integer estimate derived from the reported data.

For the currently available results, the conventional single-output ABC solution is the area baseline. The reported normalized area reduction, dual-output percentage, and structural depth gain are

$$\begin{aligned} G_{\text{area}} &= \frac{N_{\text{ABC}} - N_{\text{prop}}}{N_{\text{ABC}}} \times 100\%, \\ R_{\text{dual}} &= \frac{N_{\text{dual}}}{N_{\text{prop}}} \times 100\%, \\ G_{\text{depth}} &= \frac{D_{\text{baseline}} - D_{\text{prop}}}{D_{\text{baseline}}} \times 100\%. \end{aligned} \quad (25)$$

Here, N_{ABC} is the ABC LUT count, N_{prop} is the rounded estimated occupied-LUT count reported in the detailed tables, and N_{dual} is the reported number of packed pairs. Execution time is reported in seconds. The present experiments evaluate structural mapping depth; they do not report post-place-and-route critical delay, slack, or a calibrated commercial-device STA result.

V. EXPERIMENTAL RESULT ANALYSIS

A. Benchmark-Level Comparison

Tables III and IV report the complete per-benchmark results for both target architectures. The reference columns reproduce the proposed method’s simulation results obtained with the original benchmark setup [28], whereas the current-local-results columns report the present implementation under the experimental configuration described above. For Versal-like mapping, the current implementation achieves an average LUT-area saving of 34.96%, compared with 33.91% in the reference data, while preserving a similar dual-output percentage (58.68% versus 59.10%). The corresponding average depth improvement is 3.38%, and the mean runtime is 3.801 s. For the more restrictive UltraScale-like architecture, the average LUT-area saving increases from 22.40% to 23.39%, the dual-output percentage changes from 35.03% to 35.41%, and the current implementation improves depth by 5.13% on average with a mean runtime of 3.571 s.

The benchmark-level data also reveal substantial circuit-dependent variation. Designs such as *div*, *sin*, and *voter*

TABLE III
BENCHMARK-LEVEL RESULTS FOR VERSAL ARCHITECTURE.

Benchmark	Reference results [28]					Current results					
	LUTs	Depth	Dual (%)	Save (%)	Time (s)	LUTs	Depth	G_D (%)	Dual (%)	Save (%)	Time (s)
adder	152	51	70.39	40.16	0.07	154	51	0.00	82.41	40.27	0.04
arbiter	2467	18	10.34	9.37	2.51	2465	18	0.00	10.43	9.44	0.54
bar	448	4	14.29	12.50	0.27	448	4	0.00	14.29	12.50	0.11
cavlc	80	4	45.00	31.03	0.06	74	4	0.00	56.76	36.21	0.04
ctrl	16	2	81.25	44.83	0.01	16	2	0.00	81.25	44.83	0.01
dec	144	2	99.31	49.83	0.15	151	2	0.00	90.07	47.39	0.03
div	14155	936	83.64	35.99	274.31	12160	864	7.69	84.16	45.36	15.06
hyp	24983	4214	80.07	43.87	762.75	27689	4194	0.47	62.24	37.77	36.57
i2c	259	4	33.98	26.63	0.13	248	4	0.00	40.20	28.67	0.06
int2float	34	3	47.06	33.33	0.01	36	3	0.00	44.44	30.77	0.01
log2	5330	74	49.25	34.13	32.94	5409	71	4.05	49.75	32.15	5.30
max	513	45	52.05	33.29	0.43	507	44	2.22	51.48	33.98	0.15
mem_ctrl	8482	27	45.59	29.81	63.53	8391	25	7.41	42.97	30.12	5.49
multiplier	3790	58	56.07	36.06	22.53	3763	53	8.62	58.70	35.77	4.38
priority	123	31	74.80	41.43	0.07	119	31	0.00	75.53	44.37	0.04
router	78	7	30.77	12.36	0.01	72	7	0.00	37.06	26.29	0.01
sin	957	40	54.13	34.63	2.14	928	36	10.00	59.57	36.73	0.79
sqrt	4236	1025	72.85	25.83	13.00	3840	1024	0.10	79.06	39.81	2.60
square	2041	51	96.28	48.95	15.63	2352	50	1.96	68.79	40.95	3.42
voter	1232	20	84.82	54.29	4.63	1221	15	25.00	84.43	45.76	1.38
Average	–	–	59.10	33.91	59.76	–	–	3.38	58.68	34.96	3.80

Note: G_D denotes the depth reduction achieved by the current method relative to the reference method, calculated as $G_D = (D_{\text{ref}} - D_{\text{current}})/D_{\text{ref}} \times 100\%$. Save denotes the reduction in the LUT-area metric relative to the corresponding single-output LUT mapping baseline.

TABLE IV
BENCHMARK-LEVEL RESULTS FOR ULTRASCALE+ ARCHITECTURE.

Benchmark	LUTs	Depth	Reference results [28]			Current results					
			Dual (%)	Save (%)	Time (s)	LUTs	Depth	G_D (%)	Dual (%)	Save (%)	Time (s)
adder	183	51	40.44	27.95	0.08	181	51	0.00	69.06	29.57	0.04
arbiter	2468	18	10.29	9.33	2.00	2467	18	0.00	10.36	9.39	0.41
bar	448	4	14.29	12.50	0.23	448	4	0.00	14.29	12.50	0.09
cavlc	108	4	7.41	6.90	0.05	102	4	0.00	13.73	12.07	0.03
ctrl	18	2	61.11	37.93	0.01	18	2	0.00	61.11	37.93	0.01
dec	144	2	99.31	49.83	0.15	150	2	0.00	91.33	47.74	0.03
div	16289	943	57.97	26.34	244.69	13479	864	8.38	65.84	39.44	15.37
hyp	25007	4214	79.90	43.81	753.49	33947	4194	0.47	31.46	23.70	34.32
i2c	283	5	25.09	19.83	0.11	283	4	20.00	22.83	18.59	0.05
int2float	44	3	15.91	13.73	0.01	42	3	0.00	23.81	19.23	0.01
log2	6412	77	29.43	20.76	32.58	6249	71	7.79	28.32	21.61	4.98
max	571	45	36.60	25.75	0.39	556	44	2.22	38.25	27.67	0.13
mem_ctrl	9773	30	27.79	19.12	49.45	9326	25	16.67	28.72	22.34	4.06
multiplier	4398	56	36.02	25.80	22.85	4365	53	5.36	36.99	25.50	4.22
priority	184	31	15.76	12.38	0.07	182	31	0.00	16.25	14.79	0.04
router	76	7	17.11	14.61	0.01	81	7	0.00	20.99	16.49	0.01
sin	1198	41	24.46	18.17	2.14	1149	36	12.20	28.11	21.62	0.75
sqrt	4913	1051	28.25	13.97	11.05	5201	1024	2.57	38.26	18.49	2.10
square	2717	51	48.25	32.04	16.51	2719	50	1.96	47.58	31.73	3.43
voter	2228	20	25.22	17.33	4.77	1860	15	25.00	21.00	17.36	1.35
Average	—	—	35.03	22.40	57.03	—	—	5.13	35.41	23.39	3.57

Note: G_D denotes the depth reduction achieved by the current method relative to the reference method, calculated as $G_D = (D_{\text{ref}} - D_{\text{current}})/D_{\text{ref}} \times 100\%$. Save denotes the reduction in the LUT-area metric relative to the corresponding single-output LUT mapping baseline.

show clear depth or area improvements, whereas `hyp` remains a difficult case and uses more LUT area than the reference result under both architectures. This variation motivates reporting the complete data rather than relying only on suite averages, and it also indicates that future scoring refinements should account for benchmark structures with limited high-quality pairing alternatives.

B. Dual-Output Parameter Analysis

This subsection evaluates the joint and individual impacts of key tuning parameters on the proposed dual-output-aware

LUT mapping flow, exploring how candidate generation and pair retention limits govern the final mapping quality.

1) *Impact of Bucket and Candidate Limits:* This section evaluates the joint impact of the shared-input bucket scan limit n_{bucket} and candidate limit n_{cand} on mapping quality, as these two parameters directly dictate the candidate-generation stage. Fig. 6 illustrates the variations in mean LUT reduction and dual-output LUT ratio across the benchmark suite. Conceptually, n_{bucket} controls the local search depth within each shared-input bucket, whereas n_{cand} bounds the global candidate pool size fed into subsequent legality checking and scoring.

For both architectures, the most pronounced improvement

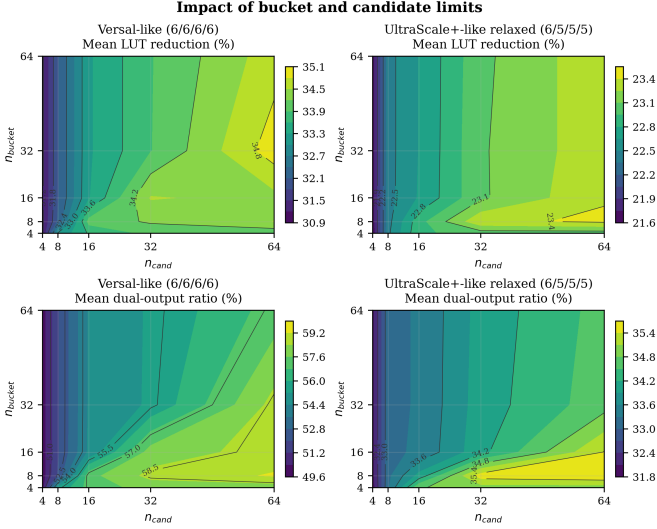


Fig. 6. Impact of n_{bucket} and n_{cand} on dual-output-aware mapping for both architectures. The top row reports mean LUT reduction and the bottom row reports mean dual-output LUT ratio; the left and right columns correspond to Versal-like and UltraScale-like architectures, respectively.

occurs as n_{cand} increases from 4 to 16. Beyond 32, the contours become nearly vertical and the incremental gain is small. At the largest tested candidate budgets, the Versal-like architecture reaches approximately 34–35% mean LUT reduction and 58–59% mean dual-output ratio, whereas the more restrictive UltraScale-like architecture reaches approximately 23–23.5% and 34–35.5%, respectively. The consistent separation between the two architectures quantifies the benefit of the larger legal dual-output support offered by the Versal-like mode.

The bucket limit has a weaker, non-monotonic effect. With a sufficiently large candidate budget, moderate values around $n_{bucket} = 8$ –16 often yield the highest dual-output ratio, while larger bucket scans provide little additional area benefit and can slightly dilute the retained candidate set. Thus, increasing the global candidate budget is useful only up to the point at which the local shared-input neighborhoods are adequately covered.

Importantly, the results demonstrate that achieving superior area efficiency relies on the synergistic optimization of both parameters under a reasonable configuration. Furthermore, the system’s global area savings exhibit a strong positive correlation with the dual-output LUT percentage. This robust positive correlation indicates that the proposed mapping framework successfully translates structural dual-output opportunities into tangible hardware reduction, rather than introducing redundant or low-quality logic duplication. This further implies that our bounded candidate search space is structurally high-yielding, confirming that an intelligent, localized clustering metric can effectively substitute for a computationally prohibitive global search while maintaining strict mapping quality.

2) *Impact of Pair Retention Limit:* This section evaluates the impact of the retained pair limit per object n_{pair} on mapping quality. This parameter controls the maximum number of high-scoring legal pair edges preserved for each LUT before

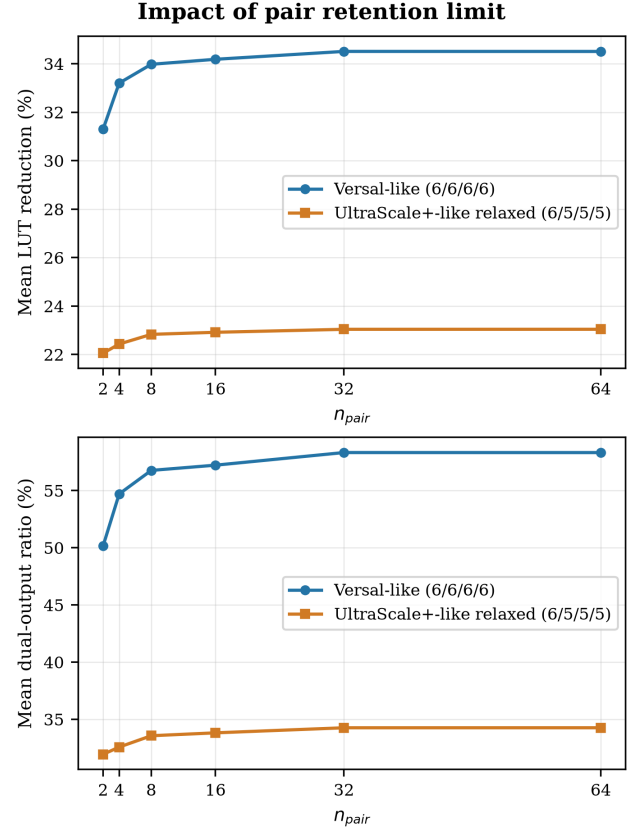


Fig. 7. Impact of n_{pair} on both target architectures: (a) mean LUT reduction and (b) mean dual-output LUT ratio.

the matching stage. Fig. 7 reports the mean LUT reduction and dual-output LUT ratio under different n_{pair} values.

The results show that mapping quality is most sensitive to n_{pair} when the retained edge budget is small. Increasing n_{pair} from 2 to 8 raises the Versal-like mean LUT reduction from about 31.3% to 34.0% and its dual-output ratio from 50.0% to about 56.7%. Over the same range, UltraScale-like improves from about 22.1% to 22.8% LUT reduction and from about 32.0% to 33.6% dual-output ratio. Thus, early pruning is more consequential for the less restrictive architecture, whose larger legal-pair space requires more alternatives during conflict resolution.

After n_{pair} reaches 16–32, both architectures saturate. At $n_{pair} = 32$, the Versal-like results are about 34.5% LUT reduction and 58.2% dual-output ratio, while UltraScale-like reaches about 23.0% and 34.3%, respectively. Increasing the limit to 64 produces no visible additional gain, showing that a compact retained graph contains the useful matching alternatives and that $n_{pair} = 16$ or 32 is a practical quality–complexity compromise.

C. ABC Original Parameter Analysis

1) *Impact of Maximum Cut Count:* This section analyzes the impact of the original ABC parameter n_{cuts} . This parameter determines the maximum number of priority cuts retained for each node during cut-based LUT mapping. A larger n_{cuts}

provides a larger cut search space, but also increases mapping runtime.

Fig. 8 compares the proposed dual-output mapper with the original ABC single-output mapper under different n_{cuts} settings. As shown in Fig. 8(a), increasing n_{cuts} improves the proposed mapper when the cut budget is small. The mean total LUT count decreases rapidly from $n_{\text{cuts}} = 2$ to $n_{\text{cuts}} = 8$, and then gradually saturates. This indicates that the dual-output mapper benefits from a sufficiently rich cut set, because more cuts provide more legal and high-quality pairing opportunities. However, after $n_{\text{cuts}} \geq 16$, the area improvement becomes marginal.

The original ABC result is much less sensitive to n_{cuts} in the same range. After the initial drop, it remains near 5600 mean LUTs, compared with approximately 3600 for Versal-like and 4180 for UltraScale-like at moderate and large cut budgets. The persistent gap between both proposed configurations and ABC shows that the area advantage mainly comes from dual-output-aware mapping and feedback rather than only from retaining more priority cuts. The additional gap between the two proposed curves is the cost of the tighter UltraScale-like legality constraints.

Fig. 8(b) shows that both flows have almost identical mapped depth across all n_{cuts} settings. The depth decreases slightly as more cuts are retained and then converges after $n_{\text{cuts}} \geq 16$. This confirms that the proposed dual-output optimization preserves the structural mapping depth of the original ABC flow while reducing the reported LUT-area metric.

Fig. 8(c) reports the runtime comparison. Runtime increases with n_{cuts} for both methods, but the proposed mapper has a higher slope because each mapping round additionally performs dual-output candidate generation, legality checking, scoring, and matching. Therefore, n_{cuts} controls an important quality-runtime trade-off. A moderate value, such as $n_{\text{cuts}} = 16$ or 32, provides most of the area benefit while avoiding the high runtime cost observed at larger cut budgets.

VI. LIMITATIONS

The present evaluation has four important limitations. First, shared-input indexing is a heuristic primary candidate source. When $S_{\text{min}} = 0$, legal disjoint-support pairs may be omitted, and the bounded matching heuristic does not guarantee a maximum-cardinality or maximum-weight pair set. Exhaustive candidate recall and optimal-matching comparisons remain necessary on tractable benchmarks to quantify this loss.

Second, the available experiments compare the proposed flow with vanilla ABC but do not yet include an ABC-plus-post-packing baseline using identical legality, scoring, and matching. Consequently, the current data demonstrate the combined benefit of dual-output packing and iterative mapping, but do not isolate the incremental contribution of feedback.

Third, timing is evaluated using structural mapping depth and output-specific logical supports. No calibrated pin-dependent LUT delays, post-routing critical-path delay, worst negative slack, or comparison with union-support timing is

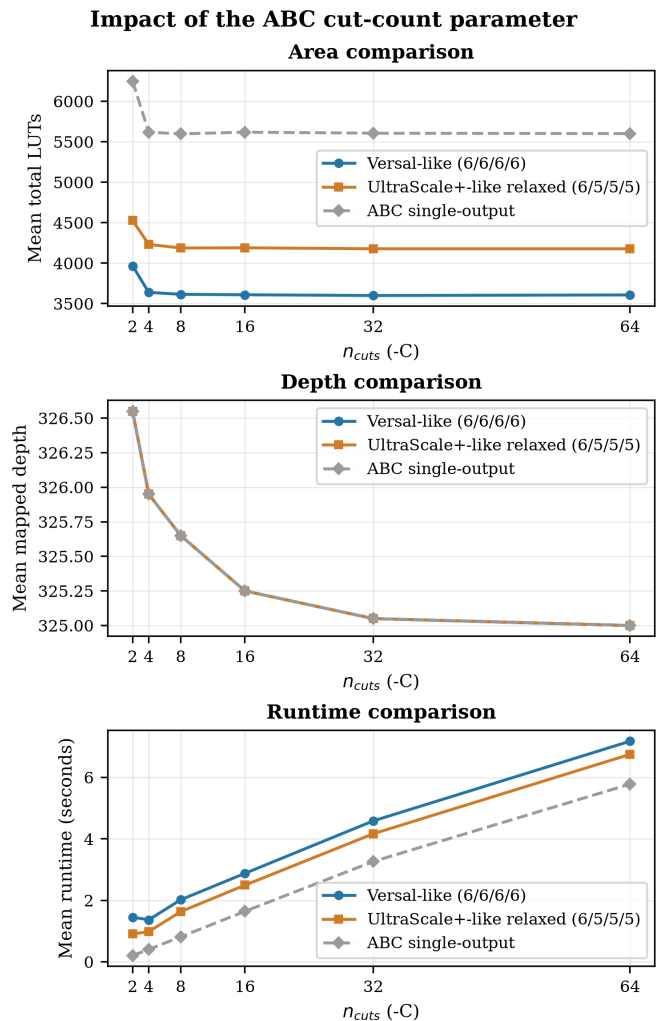


Fig. 8. Impact of the original ABC cut-count parameter n_{cuts} on ABC single-output, Versal-like, and UltraScale-like mapping: (a) mean total LUT count, (b) mean mapped depth, and (c) mean runtime.

currently available. The results therefore support preservation of mapping depth under the evaluated model, not a claim of commercial-device timing closure.

Finally, the integer LUT counts in Tables III and IV are rounded from the available source data. Directly regenerated raw single- and dual-site counts would avoid this rounding step. Repeated-run statistics, source revision, compiler options, benchmark preprocessing commands, and checksums also remain to be documented.

VII. CONCLUSION

This paper presented an iterative dual-output-aware LUT mapping framework for fracturable FPGA architecture models. The method alternates between single-output cut selection and bounded heuristic pair matching and feeds pair compatibility into subsequent area-recovery rounds. A unified legality model separates per-output support limits, physical input-union limits, optional sharing constraints, root independence, and output-specific logical timing supports.

On the currently available EPFL results, the reported LUT-area metric is reduced by 34.96% and 23.39% on average relative to vanilla ABC for the Versal-like and UltraScale-like models, respectively. Average structural-depth gains are 3.38% and 5.13%. These results show that bounded iterative pairing can expose useful resource-sharing opportunities, but they do not yet isolate feedback from post-packing or establish routed timing performance. Future work will evaluate union-aware candidate indexing, exhaustive and maximum-weight matching on tractable designs, architecture-calibrated delay models, and placement- and routing-aware dual-output mapping.

REFERENCES

- [1] J. Cong, B. Liu, S. Neuendorffer, J. Noguera, K. Vissers, and Z. Zhang, "High-level synthesis for fpgas: From prototyping to deployment," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 30, no. 4, pp. 473–491, Apr 2011. [Online]. Available: <http://dx.doi.org/10.1109/tcad.2011.2110592>
- [2] C. S. Saunders, G. Liu, Y. Yu, and W. Zhu, "Data-driven distributed analytics and control platform for smart grid situational awareness," *CSEE Journal of Power and Energy Systems*, vol. 2, no. 3, pp. 51–58, Sept 2016. [Online]. Available: <http://dx.doi.org/10.17775/cseejpes.2016.00035>
- [3] J. Timpe, K. O'Neill, D. Qendri, B. Berkane, G. Chapman, and D. Quinn, "Application of amd versal™ adaptive soc to radar space time adaptive processing in space," in *2023 European Data Handling & Data Processing Conference (EDHPC)*. IEEE, Oct 2023, pp. 1–5. [Online]. Available: <http://dx.doi.org/10.23919/edhpc59100.2023.10396329>
- [4] V. Betz, J. Rose, and A. Marquardt, *Architecture and CAD for Deep-Submicron FPGAs*. Springer US, 1999. [Online]. Available: <http://dx.doi.org/10.1007/978-1-4615-5145-4>
- [5] E. Ahmed and J. Rose, "The effect of lut and cluster size on deep-submicron fpga performance and density," in *Proceedings of the 2000 ACM/SIGDA eighth international symposium on Field programmable gate arrays*, ser. FPGA00. ACM, Feb 2000, pp. 3–12. [Online]. Available: <http://dx.doi.org/10.1145/329166.329171>
- [6] J. Lamoureux and S. Wilton, "Activity estimation for field-programmable gate arrays," in *2006 International Conference on Field Programmable Logic and Applications*. IEEE, 2006, pp. 1–8. [Online]. Available: <http://dx.doi.org/10.1109/fpl.2006.311199>
- [7] W. Li, S. Dhar, and D. Z. Pan, "Utplacef: A routability-driven fpga placer with physical and congestion aware packing," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 4, pp. 869–882, Apr 2018. [Online]. Available: <http://dx.doi.org/10.1109/tcad.2017.2729349>
- [8] M. A. Elgammal, K. E. Murray, and V. Betz, "Rlplace: Using reinforcement learning and smart perturbations to optimize fpga placement," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 8, pp. 2532–2545, Aug 2022. [Online]. Available: <http://dx.doi.org/10.1109/tcad.2021.3109863>
- [9] C. Ababei, Y. Feng, B. Goplen, H. Mogal, T. Zhang, K. Bazargan, and S. Sapatnekar, "Placement and routing in 3d integrated circuits," *IEEE Design and Test of Computers*, vol. 22, no. 6, pp. 520–531, June 2005. [Online]. Available: <http://dx.doi.org/10.1109/mdt.2005.150>
- [10] J. Cong and Y. Ding, "Flowmap: an optimal technology mapping algorithm for delay optimization in lookup-table based fpga designs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 13, no. 1, pp. 1–12, 1994. [Online]. Available: <http://dx.doi.org/10.1109/43.273754>
- [11] A. Mishchenko, S. Chatterjee, and R. Brayton, "Improvements to technology mapping for lut-based fpgas," in *Proceedings of the 2006 ACM/SIGDA 14th international symposium on Field programmable gate arrays*, ser. FPGA06. ACM, Feb 2006, pp. 41–49. [Online]. Available: <http://dx.doi.org/10.1145/1117201.1117208>
- [12] R. Brayton and A. Mishchenko, *ABC: An Academic Industrial-Strength Verification Tool*. Springer Berlin Heidelberg, 2010, pp. 24–40. [Online]. Available: http://dx.doi.org/10.1007/978-3-642-14295-6_5
- [13] T. Tan, E. Nurvitadhi, D. Shih, and D. Chiou, "Evaluating the highly-pipelined intel stratix 10 fpga architecture using open-source benchmarks," in *2018 International Conference on Field-Programmable Technology (FPT)*. IEEE, Dec 2018, pp. 206–213. [Online]. Available: <http://dx.doi.org/10.1109/fpt.2018.00038>
- [14] K. E. Murray, O. Petelin, S. Zhong, J. M. Wang, M. Eldafrawy, J.-P. Legault, E. Sha, A. G. Graham, J. Wu, M. J. P. Walker, H. Zeng, P. Patros, J. Luu, K. B. Kent, and V. Betz, "Vtr 8: High-performance cad and customizable fpga architecture modelling," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 13, no. 2, pp. 1–55, June 2020. [Online]. Available: <http://dx.doi.org/10.1145/3388617>
- [15] Z. Jiang, C. Y. Lin, L. Yang, F. Wang, and H. Yang, "Exploring architecture parameters for dual-output lut based fpgas," in *2014 24th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, Sept 2014, pp. 1–6. [Online]. Available: <http://dx.doi.org/10.1109/fpl.2014.6927470>
- [16] S. Jiang, B. Chan, K. Chung, and A. Mishchenko, "Wiremap: Fpga technology mapping for improved routability and enhanced lut merging," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 2, no. 2, pp. 1–24, June 2009. [Online]. Available: <http://dx.doi.org/10.1145/1534916.1534924>
- [17] F. Wang, L. Zhu, J. Zhang, L. Li, Y. Zhang, and G. Luo, "Dual-output lut merging during fpga technology mapping," in *Proceedings of the 39th International Conference on Computer-Aided Design, ser. ICCAD '20*. ACM, Nov 2020, pp. 1–9. [Online]. Available: <http://dx.doi.org/10.1145/3400302.3415617>
- [18] A. T. Calvino and G. De Micheli, "Technology mapping using multi-output library cells," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, Oct 2023, pp. 1–9. [Online]. Available: <http://dx.doi.org/10.1109/iccad57390.2023.10323999>
- [19] Y. Hu, V. Shih, R. Majumdar, and L. He, "Fpga area reduction by multi-output function based sequential resynthesis," in *Proceedings of the 45th annual Design Automation Conference, ser. DAC '08*. ACM, June 2008, pp. 24–29. [Online]. Available: <http://dx.doi.org/10.1145/1391469.1391478>
- [20] J.-Y. Lee, Y. Hu, R. Majumdar, L. He, and M. Li, "Fault-tolerant resynthesis with dual-output luts," in *2010 15th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, Jan 2010, pp. 325–330. [Online]. Available: <http://dx.doi.org/10.1109/aspdac.2010.5419873>
- [21] J. Shi, C. Meng, X. Wang, and W. Qian, "Quadol: A quality-driven approximate logic synthesis method leveraging dual-output luts for modern fpgas," in *2026 Design, Automation & Test in Europe Conference (DATE)*. IEEE, Apr 2026, pp. 1–7. [Online]. Available: <http://dx.doi.org/10.23919/date69613.2026.11539278>
- [22] S. Lu, L. Shang, S. Jung, Y. Zhang, and C. Pan, "A novel delay-aware packing algorithm for fpga architecture using rfet," in *2024 IEEE 67th International Midwest Symposium on Circuits and Systems (MWSCAS)*. IEEE, Aug 2024, pp. 362–366. [Online]. Available: <http://dx.doi.org/10.1109/mwscas60917.2024.10658759>
- [23] S. Lu, L. Shang, S. Jung, Q. Liang, and C. Pan, "A novel rfet-based fpga architecture based on delay-aware packing algorithm," *IEEE Transactions on Emerging Topics in Computing*, vol. 13, no. 3, pp. 1230–1241, July 2025. [Online]. Available: <http://dx.doi.org/10.1109/tetc.2025.3572712>
- [24] S. Lu, L. Shang, S. Jung, Y. Zhang, Q. Liang, and C. Pan, "Technology/system co-optimization for fpga using emerging reconfigurable logic device," *ACM Journal on Emerging Technologies in Computing Systems*, vol. 21, no. 3, pp. 1–21, July 2025. [Online]. Available: <http://dx.doi.org/10.1145/3750730>
- [25] P.-E. Gaillardon, X. Tang, G. Kim, and G. De Micheli, "A novel fpga architecture based on ultrafine grain reconfigurable logic cells," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 23, no. 10, pp. 2187–2197, Oct 2015. [Online]. Available: <http://dx.doi.org/10.1109/tvlsi.2014.2359385>
- [26] L. Amarú, P.-E. Gaillardon, and G. De Micheli, "Bds-maj: a bdd-based logic synthesis tool exploiting majority logic decomposition," in *Proceedings of the 50th Annual Design Automation Conference, ser. DAC '13*. ACM, May 2013, pp. 1–6. [Online]. Available: <http://dx.doi.org/10.1145/2463209.2488792>
- [27] S. Beamer, K. Asanović, and D. Patterson, "Direction-optimizing breadth-first search," *Scientific Programming*, vol. 21, no. 3-4, pp. 137–148, 2013. [Online]. Available: <http://dx.doi.org/10.1155/2013/702694>
- [28] S. Lu, L. Shang, Q. Qu, S. Jung, Q. Liang, and C. Pan, "An efficient multi-output lut mapping technique for field-programmable gate arrays," *Electronics*, vol. 14, no. 9, p. 1782, Apr 2025. [Online]. Available: <http://dx.doi.org/10.3390/electronics14091782>